Mobile Apps

BACKEND



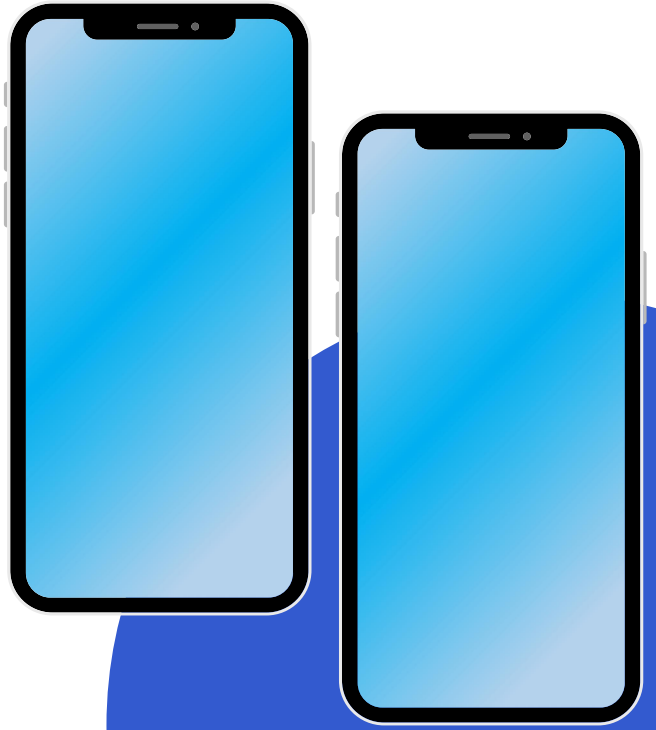


Welcome To Our Application

- Mobile apps often need more than just frontend (UI/UX) – they require data storage, user authentication, and real-time features.
- Backend acts as the "brain" behind the app, handling server-side logic.
- Today, we'll explore backend concepts, tools, and focus on a lightweight recommendation.

What is a Backend?

- Definition: The backend is the server-side part of an application that manages data, logic, and communication between the app and external services.
- Components:
- Servers: Host the app's logic.
- Databases: Store user data, app content.
- APIs: Interfaces for frontend to interact with backend (e.g., RESTful or GraphQL).
- Mobile Context: For apps like social media or e-commerce, backend handles user logins, data syncing across devices.





Why Do Mobile Apps Need a Backend?

- Reasons:
- Data Persistence: Store user profiles, posts, or scores beyond the device.
- Security: Handle authentication (login/signup) and protect sensitive data.
- Scalability: Support multiple users without crashing the app.
- Real-Time Features: Push notifications, live updates (e.g., chat apps).
- Without Backend: Apps are limited to local storage, offline-only, or insecure.

Types of Backend Approaches for Mobile Apps



- Traditional Backend: Custom servers (e.g., Node.js, Django) – powerful but complex to set up.
- Backend as a Service (BaaS): Cloud-based, no-server management (e.g., Firebase, AWS Amplify).
- Serverless: Functions run on demand (e.g., AWS Lambda) – efficient for small apps.

Pros/Cons Table:

Type	Pros	Cons
Traditional	Full control	Time-consuming setup
BaaS	Quick, lightweight	Less customization
Serverless	Scalable, cost-effective	Learning curve for events

What is BaaS?

Backend-as-a-Service Overview

Core Concept

A ready-made backend you use through an **SDK** instead of building and maintaining your own custom server infrastructure.

Typical Features

- Database Management
- Authentication & Security
- Cloud File Storage
- Serverless Functions
- Realtime Updates

Analogy

Using BaaS is like **renting a fully equipped commercial kitchen** instead of constructing one from scratch. You bring the recipe, they provide the infrastructure.

Why BaaS for Flutter Apps?

Key advantages for modern cross-platform development

Accelerated Velocity

Faster development allows you to stay focused on building exceptional UI and rich client features.

Zero DevOps Overhead

No server setup, infrastructure configuration, or ongoing backend maintenance required.

Seamless Elasticity

Scales automatically with your user growth, handling traffic spikes without manual intervention.

Capital Efficient

Significantly lower upfront costs, making it the ideal choice for students, startups, and rapid MVPs.

Seamless Elasticity in Action

How BaaS seamlessly manages unexpected traffic and user growth spikes

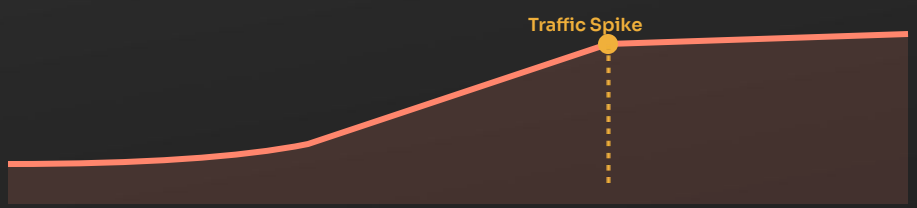
Automated Infrastructure Scaling

BaaS platforms monitor incoming load continuously. When traffic bursts occur, additional cloud instances instantiate automatically in the background—no human intervention required.

Does BaaS handle traffic spikes?

Yes, absolutely!

Your app delegates load balancing, database connection pooling, and server provisioning directly to the provider's infrastructure.



Key Benefits for Growth:

- Zero server crash during marketing bursts
- Pay only for active compute usage
- Instant global distribution via CDN edge

Firebase vs Supabase

Quick Comparison: Google's NoSQL BaaS vs Open Source SQL BaaS

Firebase (Google)

Database: NoSQL (Firestore / Realtime DB)

Ecosystem: Proprietary closed-source cloud

Key Features: Crashlytics, Analytics, Remote Config, ML Kit

Querying: Document-based, limited complex queries

Trade-off: Matured ecosystem, but higher vendor lock-in

Supabase (Open Source)

Database: Relational SQL (PostgreSQL)

Ecosystem: Open-source & self-hostable

Key Features: Auto-generated APIs, Vector embeddings, Edge functions

Querying: Full SQL power, complex joins, relational integrity

Trade-off: Zero lock-in & SQL flexibility, but smaller addon ecosystem

Core Services Compared

Firebase vs Supabase feature mapping

Service	Firebase	Supabase
Database	Firestore (NoSQL)	Postgres (SQL)
Auth	Firebase Auth	Supabase Auth
Storage	Cloud Storage	Supabase Storage
Functions	Cloud Functions	Edge Functions
Realtime	Firestore listeners	Realtime channels

Database Model: NoSQL vs SQL

Data structuring and when each model fits best

Firestore (NoSQL)

Structure

Documents and collections, flexible schema, denormalized data

When It Fits

Ideal for simple, rapidly changing, or hierarchical data where fast reads and flexible schemas are prioritized.

Supabase (SQL)

Structure

Tables, relations, joins, and relational constraints

When It Fits

Best suited for structured, highly relational data requiring strict consistency and complex querying capabilities.

SUMMARY

- **NoSQL (Firestore):** Choose for unstructured data and rapid scaling
- **SQL (Supabase):** Choose for relational integrity and complex joins

How Firestore Stores Data

Document-based NoSQL model with dynamic schemas

Collection & Document Concept

Firestore stores documents (JSON-like objects) inside collections.

- **Self-Contained:** Each document carries its own structure and nested data.
- **No Enforced Schema:** Two documents in the exact same collection can have completely different fields.
- **Flexible Evolution:** Easily add or modify fields as application requirements grow without complex migrations.

Document Structure Example

```
// Path: users/{userId}

{
  "name": "Ana",
  "email": "ana@mail.com",
  "address": {
    "city": "Davao",
    "zip": "8000"
  },
  "favoriteTags": [
    "flutter",
    "dart"
  ]
}
```

Firestore Query Examples

From basic field filtering to chaining multiple constraints and listeners

Firestore Client SDK (Web/Mobile)

```
// Basic filter query
const q = query(
  collection(db, 'users'),
  where('email', '==', 'ana@mail.com')
);

// Execute query and fetch docs
const querySnapshot = await getDocs(q);
querySnapshot.forEach((doc) => {
  console.log(doc.id, '=>', doc.data());
});
```

Chained Queries & Real-Time

```
// Compound query with array filtering
const complexQuery = query(
  collection(db, 'users'),
  where('favoriteTags', 'array-contains',
  'flutter'),
  orderBy('name'),
  limit(10)
);

// Real-time document updates
onSnapshot(complexQuery, (snapshot) => {
  handleUpdates(snapshot.docs);
});
```

How Supabase Stores Data

Relational PostgreSQL model with strict schemas and foreign keys

Relational Table Concept

Supabase uses Postgres tables with fixed columns and data types.

- **Normalized Data:** Data is split across discrete tables to prevent redundancy.
- **Foreign Keys:** Records are connected explicitly through relational foreign keys.
- **Strict Schema:** Ensures high consistency, transactional integrity, and structured querying capabilities.

Relational Schema Example

```
-- SQL Table Definitions
users (id, name, email)
  PK: id

addresses (id, user_id, city, zip)
  FK: user_id → users.id

tags (id, name)
  PK: id

user_tags (user_id, tag_id)
  FK: user_id → users.id
  FK: tag_id → tags.id
```

What Supabase Offers

Core capabilities powered by an open-source relational engine

PostgreSQL Core

- **Relational Foundation**

At its core, Supabase uses PostgreSQL, a powerful open-source relational database system.

- **Proven Reliability**

Built on decades of enterprise-grade database stability and performance.

SQL Power

- **Standard Interface**

Allows you to manage and query data using standard SQL directly.

- **Universal Language**

Leverage SQL, the most popular and widely adopted language for database interactions.

Open Source Architecture

- **Community-Driven**

Fully open-source stack avoiding vendor lock-in and allowing self-hosting.

- **Extensible Ecosystem**

Seamlessly integrate native Postgres extensions, plugins, and custom tooling.

Supabase & Postgres Query Examples

From standard SQL joins to real-time client SDK filters

Standard SQL Query (Postgres)

```sql
// Complex relational query with JOINs
SELECT users.name, addresses.city
FROM users
JOIN addresses ON addresses.user_id = users.id
WHERE users.email = 'ana@mail.com';

// Insert structured relational data
INSERT INTO users (name, email)
VALUES ('Ana', 'ana@mail.com');
```

Supabase Client SDK (JS/Dart)

```javascript
// Fetch user with nested address relation
const { data, error } = await supabase
  .from('users')
  .select('name, addresses(city)')
  .eq('email', 'ana@mail.com');

// Subscribe to real-time changes
supabase.channel('public:users')
  .on('postgres_changes', { event: '*' },
  handleChanges)
  .subscribe();
```

Key Differences: Firestore vs PostgreSQL

Comparing document NoSQL and relational database paradigms

Feature	Firestore (NoSQL)	PostgreSQL (Relational)
Schema	Flexible schematic structure. Add fields at any time without downtime.	Strict schemas requiring explicit migrations. Automatically rejects invalid data.
Relationships	No native JOIN support. Requires multiple document reads or denormalized structures.	Built-in JOINs, foreign keys, and relational constraints.
Querying	Simple and fast queries. Limited aggregations and filter combinations.	Full SQL capabilities: complex JOINs, GROUP BY, aggregations, and search.
Data Duplication	Common to duplicate data to optimize reads, requiring multi-document updates.	Normalized storage: data stored once and referenced via keys/joins.
Scaling Model	Automatic horizontal scaling for reads/writes with minimal tuning required.	Traditional database scaling: requires active indexing, query tuning, and management.

When Firestore Fits Best

Optimal use cases for document-based NoSQL architecture

Simple Data & Speed

- **Simple, Self-Contained Data**

  Ideal for chat messages, notes, to-do items, user profiles, and app settings.

- **Fast Prototyping**

  Perfect when the schema keeps changing rapidly in early development stages.

Read Efficiency

- **Read-Heavy Screens**

  Load one complete document and display it directly with zero JOIN operations needed (e.g., social feeds, product details pages).

Real-Time & Mobile

- **Offline-First Apps**

  Firestore's robust offline caching capability offers a massive advantage for mobile applications.

- **Live Reactive Updates**

  Listening to document or collection changes real-time is seamless (e.g., Flutter with StreamBuilder).

When Supabase (SQL) Fits Best

Optimal use cases for relational PostgreSQL architecture

Relational & Integrity

- **Structured Data Relationships**

E-commerce (users, orders, products), school systems (students, grades), and booking platforms.

- **Strict Data Integrity**

Foreign keys, unique constraints, and strong typing stop bad data at the database level.

Analytics & Queries

- **Powerful Analytics**

Totals, averages, and top-N queries running natively in SQL instead of complex client code.

- **Advanced Filtering & Search**

Complex conditions, multi-column sorting, relational joins, and full-text search capabilities.

Flexibility & Control

- **Avoid Vendor Lock-in**

Built entirely on open-source Postgres, enabling full data export or self-hosting anytime.

- **Enterprise Ready**

Ideal for financial, inventory, and core application engines where reliability is critical.



Supabase

What Makes Supabase Stand Out?

The open-source Firebase alternative built on PostgreSQL

Open-Source Core

- **Firebase Alternative**

  Built on open standards to avoid vendor lock-in with full data ownership.

- **SQL Familiarity**

  Leverages standard PostgreSQL syntax, tools, and decades of ecosystem reliability.

All-in-One Suite

- **Integrated Services**

  Combines Postgres DB, Auth, Realtime subscriptions, and File Storage out of the box.

- **Few-Click Setup**

  Provision complete production-ready backend infrastructure in seconds.

Zero Server Management

- **Fully Managed**

  No infrastructure overhead, patching, or devops setup required to launch.

- **Seamless Scaling**

  Focus purely on frontend application logic while the platform handles capacity.



Supabase Features

Feature 01: Postgres Database

Every project starts with a full Postgres database, and the other services build on top of it.

CORE DB

Full Postgres

• Uncompromised Engine

A real relational database with no compromises, ensuring maximum reliability.

• Triggers & Functions

Leverage robust native triggers and stored procedures right inside your database.

Extensions

• Powerful Add-ons

Turn on extra capabilities easily with a single click or command.

• pgvector & pg_cron

Store AI embeddings seamlessly or run scheduled background jobs directly in Postgres.

Studio & CLI

• Dashboard Studio

Browse tables, edit schema, and run raw SQL queries effortlessly from the dashboard.

• Local CLI Tools

Manage database migrations and local development with the powerful command line.

PostgreSQL

An object-relational database combining classic relational features with powerful extensibility.

• Tables, Rows & Columns

Data lives in structured tables composed of rows and columns, strictly linked together through keys to organize information seamlessly.

• SQL Queries & Joins

Query data efficiently using standard SQL syntax and seamlessly combine multiple related tables using powerful joins.

• Strict Data Constraints

Comprehensive constraints—including Primary Keys, Foreign Keys, Unique rules, and Check conditions—protect data integrity.

• ACID Transactions

Transactions adhere to strict ACID principles, guaranteeing that operation updates either fully succeed or completely fail without side effects.

Postgres: Beyond Relational

Advanced extensibility features that power modern data architectures

• Custom Types

Define composite types, enums, domains, and ranges to use directly as column types.

• Rich Built-in Types

Native JSONB, arrays, UUID, and timestamptz mix structured and semi-structured data seamlessly.

• Extensions Ecosystem

Plug in pgvector, PostGIS, pg_trgm, and pg_cron without modifying your application core.

• Advanced Indexing

GIN, GiST, and BRIN indexes optimized for JSON, full-text search, spatial, and massive tables.

• Custom In-DB Logic

Write powerful functions, triggers, and custom operators in PL/pgSQL and other languages.

• Advanced SQL

Built-in CTEs, window functions, materialized views, and Row Level Security (RLS).

• High-Concurrency MVCC

Multi-Version Concurrency Control guarantees readers and writers never block each other.

• Table Inheritance

Object-oriented data modeling capability where child tables inherit columns from parents.

Relational vs. Extensible Engine

How Postgres transcends traditional database boundaries to unify multi-model workloads

a relational database stores data in tables, while Postgres also lets you define new data types, behavior and features, so one database can handle relational, JSON, search, geospatial and vector workloads.

Grouped Values: Columns vs Custom Type

Comparing plain relational SQL structures with custom composite types in PostgreSQL

• Plain Relational SQL

Flat structure storing address fields as separate columns in the table.

```
-- Table Definition
CREATE TABLE customers (
id bigint PRIMARY KEY,
name text,
home_street text,
home_city text,
home_zip text
);

-- Querying Data
SELECT name, home_city
FROM customers;
```

• PostgreSQL Composite Type

Encapsulates related fields into a reusable custom domain type.

```
-- Type & Table Definition
CREATE TYPE address AS (
street text, city text, zip text
);

CREATE TABLE customers (
id bigint PRIMARY KEY,
name text,
home address
);

-- Querying Data
SELECT name, (home).city
FROM customers;
```

Feature 02: Auto-generated APIs

Your tables become an API generated from your database schema, with no server code to write.

• REST API

PostgREST turns your database schema directly into a fully functional, secure REST API for reading and writing data without writing backend logic.

• GraphQL API

The **pg_graphql** extension adds a powerful, auto-generated GraphQL API natively on top of your PostgreSQL schema.

• Client Libraries

Seamlessly connect from modern frontends using official, feature-complete client libraries:

- **Flutter**
- **JavaScript**
- **Python**
- **Swift**

Feature 03: Authentication

Sign-up and sign-in come built in, with users stored right next to your data.

• Many Sign-in Methods

Flexible authentication support for email, social, phone, and passwordless login, plus integration with any OAuth2 or OIDC provider.

• Customizable Flows

Auth hooks (beta) allow you to run custom database functions or Edge Functions directly during the sign-in process.

• Stored in Postgres

User data lives directly in the **auth schema** of your own database, ensuring complete control rather than relying on a third party.

Feature 04: Row Level Security

Postgres policies decide which rows each user can read or change.

• Policies in SQL

Write rules per table that check who is asking and which rows they touch.

• Deny until allowed

With RLS on and no policies, the **API returns nothing** until you add one.

• Tied to Auth

Policies can use the **signed-in user's identity**, and Storage follows the same rules.

Feature 05: Storage

Cloud Store and serve files like images, video and documents, protected by your Postgres access rules.

OBJECTS

Buckets and files

- **Bucket Organization**

Organize uploads in distinct buckets for different media types and use cases.

- **Access Control Policies**

Control access with the same Postgres Row Level Security policies as your data.

CDN and transforms

- **Fast Delivery**

Serve files globally through an integrated Content Delivery Network for low latency.

- **Dynamic Resizing**

Resize and optimize images on the fly via simple URL parameters.

Uploads and S3

- **Resumable Uploads**

Handle large files reliably with background resumable multipart uploads.

- **S3 Compatibility**

Use standard S3-compatible APIs and tools seamlessly with existing workflows.

Feature 06: Realtime

Push live updates to connected clients over WebSockets.

Postgres Changes

- **Live Database Stream**

Stream inserts, updates and deletes from your tables to subscribed clients.

Broadcast

- **Low-latency Messaging**

Send low-latency messages between clients, even from the database.

Presence

- **User State Sync**

Share and sync state, such as who is online in a room.

Feature 07: Edge Functions

Run custom server-side TypeScript without managing servers.

Deno and TypeScript

- **Deno Runtime**

Functions run on the Deno runtime, with npm module support.

Close to users

- **Global Deployment**

Deploy globally, or invoke a function in a region near your database.

Pre-wired

- **Zero Config**

Environment variables for your database, auth and storage come ready to use.

Feature 08: Database Webhooks

React to changes in your tables by calling out to other services.

LIGHTNING

Table events

- **Trigger Events**

Fire when rows are inserted, updated or deleted.

Functions or HTTP

- **Event Delivery**

Send the event to an Edge Function or any external HTTP endpoint.

Still in beta

- **Beta Status**

The features page lists webhooks as beta, so check the docs for current limits.

Feature 10: Cron

Clock Schedule recurring jobs from inside your database.

Powered by pg_cron

- **Postgres Extension**

The scheduler runs as a Postgres extension, right next to your data.

Recurring work

- **Automated Tasks**

Handle cleanups, reports and retries on a schedule.

Pairs with functions

- **Edge Integration**

Use pg_net to call an Edge Function on a schedule when SQL alone is not enough.

Feature 11: Queues

Durable message queues with guaranteed delivery, built on Postgres.

PAPERPLANE

Built on pgmq

- **Postgres Extension**

Messages live in your own Postgres database, alongside your other data.

Guaranteed delivery

- **Durable Messaging**

Messages are durable, so queued work is not lost between steps.

Background jobs

- **Offload Tasks**

Offload slow work and retry failures, for example when generating embeddings.



Supabase Setup

Setting Up Supabase in Flutter

Four simple steps to integrate PostgreSQL backend into your cross-platform app

1

Add Dependency

Add the official Supabase Flutter package to your **pubspec.yaml** file:

```
dependencies:
supabase_flutter:
^2.0.0
```

Or run in your terminal:

```
flutter pub add
supabase_flutter
```

2

Initialize Supabase

Initialize the client in your **main.dart** before running the app:

```
void main() async {
WidgetsFlutterBinding
.ensureInitialized();
await Supabase.
initialize(
url: 'YOUR_URL',
anonKey: 'KEY',
);
runApp(MyApp());}
```

3

Access Client Instance

Get a reference to the global Supabase client instance anywhere in your app:

```
final supabase =
Supabase.instance
.client;
```

Use this client to handle authentication, database queries, and storage operations seamlessly.

4

Perform DB Operations

Query data directly from your Flutter widgets using async/await:

```
final data = await
supabase
.from('notes')
.select();
```

Real-time listening:

```
.stream(primaryKey:
['id'])
```

Setting Up Supabase in Flutter

Four simple steps to integrate PostgreSQL backend into your cross-platform app

1

Add Dependency

Add the official Supabase Flutter package to your **pubspec.yaml** file:

```
dependencies:
supabase_flutter:
^2.0.0
```

Or run in your terminal:

```
flutter pub add
supabase_flutter
```

2

Initialize Supabase

Initialize the client in your **main.dart** before running the app:

```
void main() async {
WidgetsFlutterBinding
.ensureInitialized();
await Supabase.
initialize(
url: 'YOUR_URL',
anonKey: 'KEY',
);
runApp(MyApp());}
```

3

Access Client Instance

Get a reference to the global Supabase client instance anywhere in your app:

```
final supabase =
Supabase.instance
.client;
```

Use this client to handle authentication, database queries, and storage operations seamlessly.

4

Perform DB Operations

Query data directly from your Flutter widgets using async/await:

```
final data = await
supabase
.from('notes')
.select();
```

Real-time listening:

```
.stream(primaryKey:
['id'])
```

2. Initialize Supabase in main.dart

Set up authentication and database connections before your Flutter app runs

main.dart Setup

```
import 'package:flutter/material.dart';
import 'package:supabase_flutter/ ... ';

Future<void> main() async {
WidgetsFlutterBinding.ensureInitialized();

await Supabase.initialize(
url: 'https://YOUR_[link removed]',
anonKey: 'YOUR_ANON_KEY',
);

runApp(const MyApp());
}

final supabase = Supabase.instance.client;
```

Project Credentials

Get your unique URL and Key from:
• **Project Settings → API**

The supabase global shortcut provides instant reference to the client anywhere in your app structure.

Security Best Practices

▪ Public Key Only: Use only the public (anon / publishable) key in your client application.

▪ Never Expose Secret Keys: Never embed the service-role or secret key in client code.

3. Query Data in Dart

Perform CRUD operations directly using the Supabase Flutter client

CRUD Operations

```dart
// Read
final todos = await supabase
  .from('todos')
  .select();

// Insert
await supabase
  .from('todos')
  .insert({'task': 'Learn Supabase'});

// Update
await supabase
  .from('todos')
  .update({'done': true})
  .eq('id', 1);

// Delete
await supabase
  .from('todos')
  .delete()
  .eq('id', 1);
```

Row Level Security (RLS)

- **Access Control:** With RLS enabled, these queries will return empty results or fail until proper policies are defined in Supabase.
- **Action Required:** Ensure you add policies for SELECT, INSERT, UPDATE, and DELETE operations in your dashboard.

Query Execution Tips

- **Async/Await:** Database queries are asynchronous and require using `await` or handling Futures.
- **Method Chaining:** Filters like `.eq()`, `.gt()`, or `.order()` can be chained directly onto the query.

FLUTTER & DART

Querying Data with Supabase

A Cheatsheet

Supabase Update & Delete Operations (Flutter/Dart)

Safely modifying and removing PostgreSQL records with required filters

Updating Data (.update)

```dart
// Update single row & return response
final response = await supabase
.from('products')
.update({'price': 99})
.eq('id', 5)
.select()
.single();
```

Updates columns for rows matching the filter. Use `.select().single()` to return the updated record.

Deleting Data (.delete)

```dart
// Delete row & return deleted record
final deleted = await supabase
.from('products')
.delete()
.eq('id', 5)
.select();
```

Deletes target rows specified by filter. Appending `.select()` yields the deleted row details.

Critical Safety Rule: Always Target with Filters

Executing `.update()` or `.delete()` without filter methods like `.eq()` will affect **all rows** in the table unless restricted by RLS policies.

Best Practice: Always chain `.eq()`, `.inFilter()`, or other query filters immediately after `.update()` or `.delete()` to prevent accidental mass mutations. Chain `.select()` to receive modified payload data directly.

Supabase Filter Cheat Sheet (Flutter/Dart)

Common query filter methods for refining PostgreSQL data fetches

Exact Match (.eq)

```dart
// Equal to specific value
await supabase.from('products')
.select()
.eq('category_id', 3);
```

Filters rows where `category_id` equals `3`.

Greater Than (.gt)

```dart
// Strictly greater than
await supabase.from('products')
.select()
.gt('stock', 0);
```

Filters rows where `stock` is strictly `> 0`.

Range Filtering (.gte & .lte)

```dart
// Greater/Equal & Less/Equal (Chained)
await supabase.from('products')
.select()
.gte('price', 10).lte('price', 50);
```

Combines filters for bounds: `10 ⩽ price ⩽ 50`.

Pattern Matching (.ilike)

```dart
// Case-insensitive substring match
await supabase.from('products')
.select()
.ilike('name', '%shoe%');
```

Matches `name` containing 'shoe' regardless of case.

List Inclusion (.inFilter)

```dart
// Match any value in a list (SQL IN operator)
await supabase.from('products').select().inFilter('category_id', [1, 2]);
```

Filters rows where `category_id` is either `1` or `2`. Essential for multiple selection queries.

Supabase Ordering & Pagination (Flutter/Dart)

Essential query methods for sorting and paginating PostgreSQL dataset results

Ordering Results (.order)

```dart
// Sort by column explicitly
await supabase.from('products')
.select()
.order('price', ascending: true);
```

Sorts returned rows by `price` in ascending order. Set `ascending: false` for descending.

Range Pagination (.range)

```dart
// Paginate using zero-based index
await supabase.from('products')
.select()
.range(0, 19);
```

Fetches a subset of records from index `0` to `19` (retrieves exactly 20 items for page 1).

Chained Query Pattern (Filters + Order + Paginate)

```dart
// Recommended execution order: Apply filters first, sort, then paginate
final response = await supabase
  .from('products')
  .select()
  .eq('category_id', 3)  // 1. Filter
  .order('price', ascending: true)  // 2. Order
  .range(0, 19);  // 3. Paginate (rows 0 to 19)
```

Always chain `.order()` and `.range()` after your filter methods (`.eq()`, `.gt()`, etc.) to ensure accurate page offsets and deterministic query ordering.

Company Name

Let's discuss any doubts or concerns.

If something comes up later, contact name@example.com

Any questions? Ask away!

Year/Quarter/Month


Thank you